

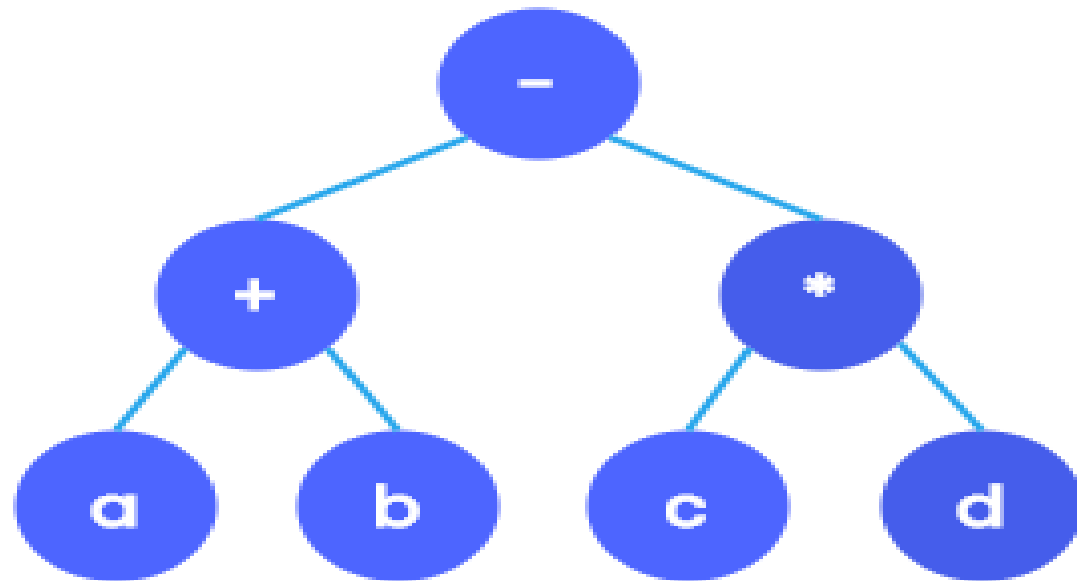
Expression Trees

- An expression tree is a tree built up from the operands as the leaves of binary tree and operators as the non -leaves of binary tree.
- It is a special kind of binary tree in which
 - Each leaf node contains single operand
 - Each non-leaf node contains a single binary operator
 - The left and right subtrees of an operator node represent sub-expression that must be evaluated before applying the operator at the root of the subtree.

Idea

- Levels in a binary expression tree represent the precedence of operators.
- Operators at the lower level must be evaluated first
- Then the operators at the next level and so on and at the last operator at the root node is applied and there by the expression is evaluated.

Ex : $X = (a + b) - (c * d)$



Constructing an Expression Tree

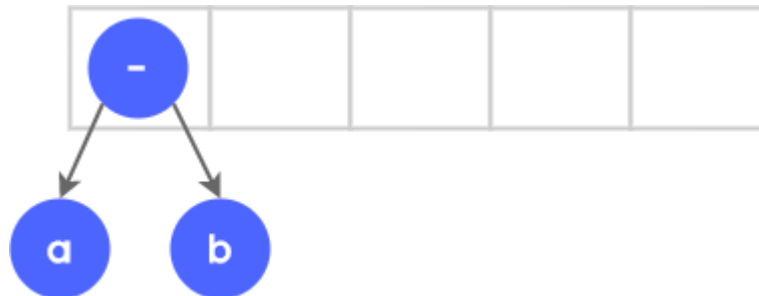
- Convert the given Expression into Postfix form.
- Repeat the following steps:
 - If the **character is an operand**, create a node and **PUSH** it on to the stack.
 - If the **character is an operator**, **POP** two values from the stack and make a node keeping the operator as root node and the two items as the left and the right child.
 - Push the newly generated node into the stack.

Consider the Post fix Expression: **a b - c ***

- Step 1: First two characters are operands, PUSH it onto the stack.



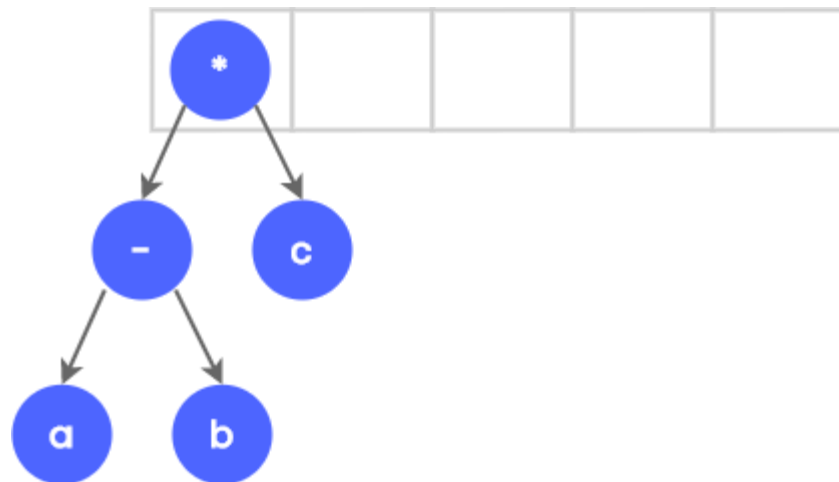
- Step 2: Next, “-” Symbol is read, “a” and “b” are popped from the stack and is added as the child of node “-” . A pointer to the new node is now pushed onto the stack.



- Step 3: Next, “c” is read and is pushed onto the stack.



- Step 4: Next, “*”, so pop “c” and “-” from the stack and add it as child of “*”



Construction of Expression Tree

```
ConstructExpressionTree(postfix[])
```

```
{
```

```
    // Traverse through every character of input expression
```

```
    for (int i=0; i<strlen(postfix); i++)
```

```
    {
```

```
        // If operand, simply push into stack
```

```
        if (!isOperator(postfix[i]))
```

```
        {
```

```
            t = newNode(postfix[i]);
```

```
            stack.push(t);
```

```
        }
```

else // If operator

{

t = newNode(postfix[i]);

// Pop two top nodes

t1 = stack.top(); **// Store top**

stack.pop(); **// Remove top**

t2 = stack.top();

stack.pop();

// make them children to the created node

t->right = t1;

t->left = t2;

// Add this subexpression to stack

stack.push(t);

}

}// End of For Loop

// only element will be root of expression tree

t = stack.top();

stack.pop();

return t;

}